



# Módulo III

## Especialización en IA Generativa y Machine Learning OPS

### Herramientas para la Inteligencia Artificial Generativa



01

Introducción a  
LangChain y  
LlamaIndex

02

Fundamentos  
LangChain

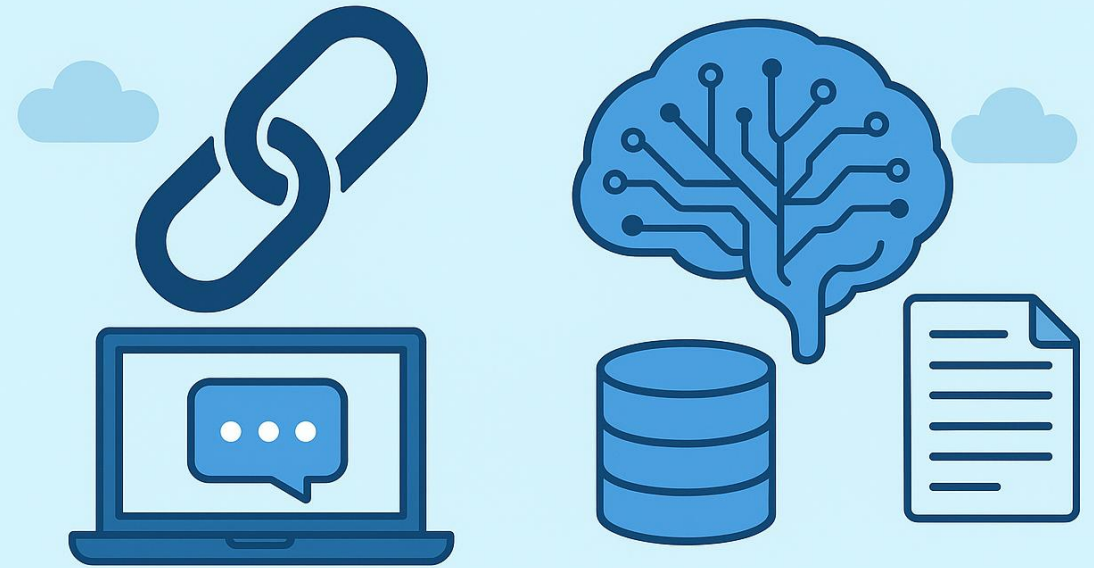
03

Herramientas RAG:  
LangChain -  
LlamaIndex

04

Herramientas RAG:  
Vector Stores

# LANGCHAIN LLAMAINDEX





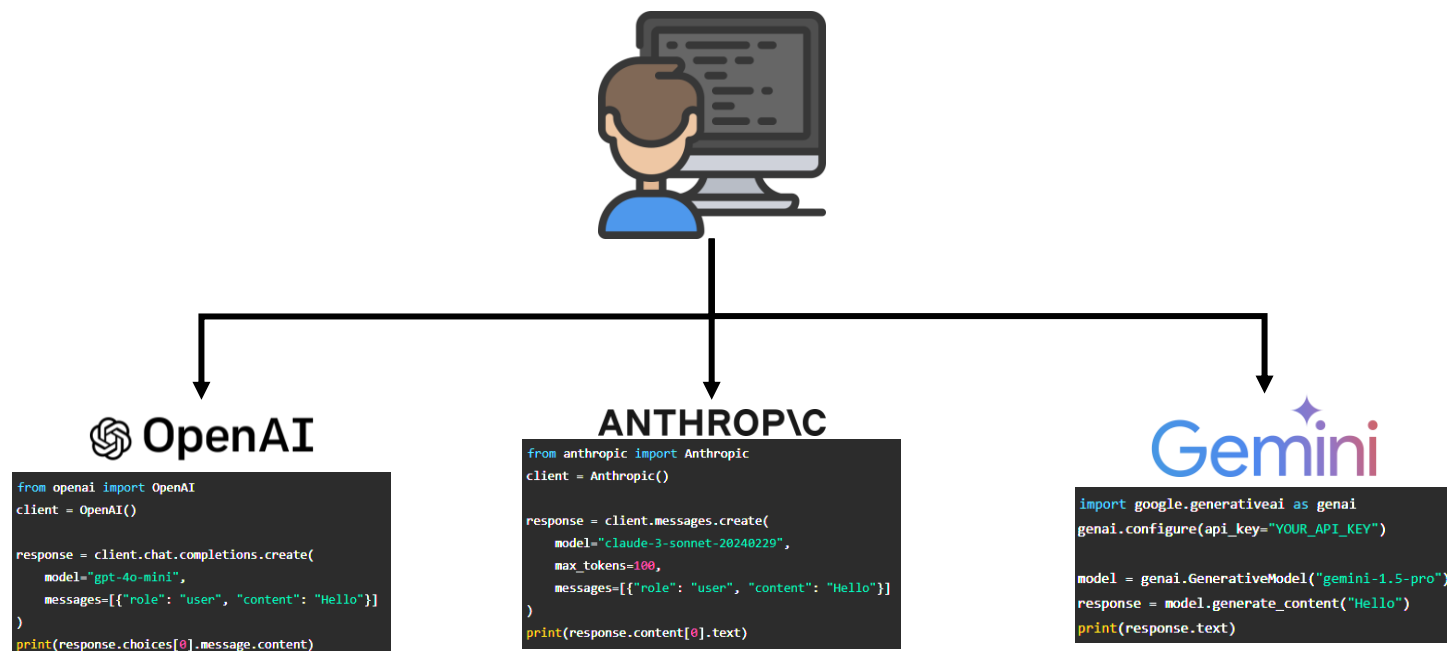
**01**

Introducción a  
LangChain y  
LlamaIndex

## ¿Por qué usar herramientas para GenAI?



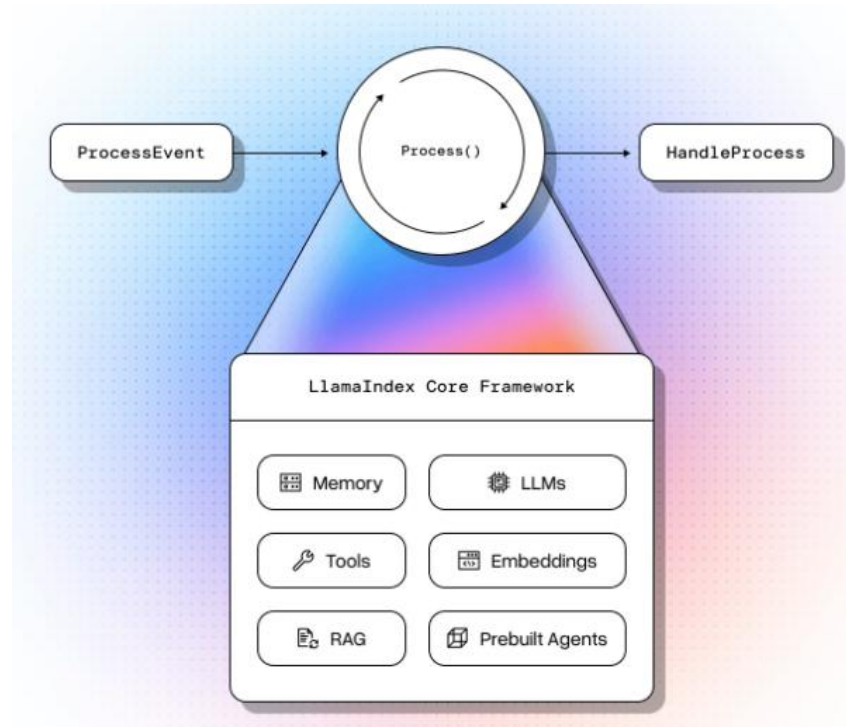
Usar modelos de distintos proveedores exige trabajar con librerías y **sintaxis diferentes**, lo que aumenta la **complejidad**, obliga a aprender **varias interfaces** y dificulta integrar o cambiar de modelo en una misma aplicación.



# LlamaIndex



**LlamaIndex** es un framework **orientado a desarrolladores** que acelera la implementación de aplicaciones de **IA generativa**, ofreciendo abstracciones de alto y bajo nivel confiables. Está optimizado para agentes, **RAG**, flujos de trabajo personalizados e integraciones.

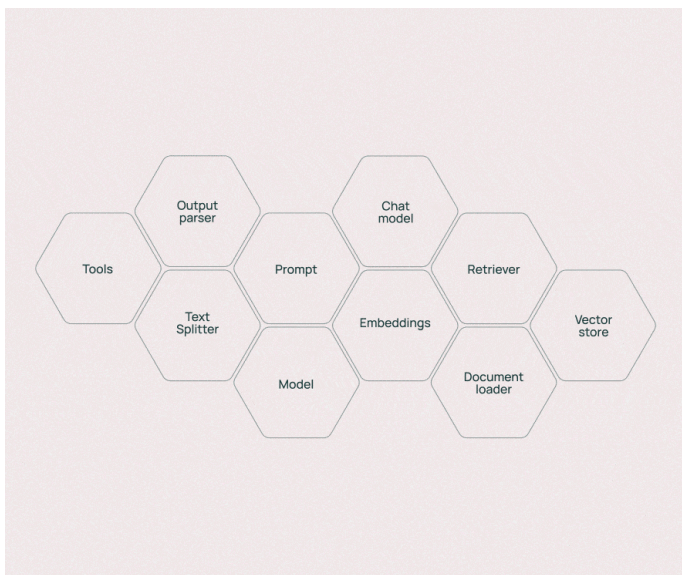


# LangChain

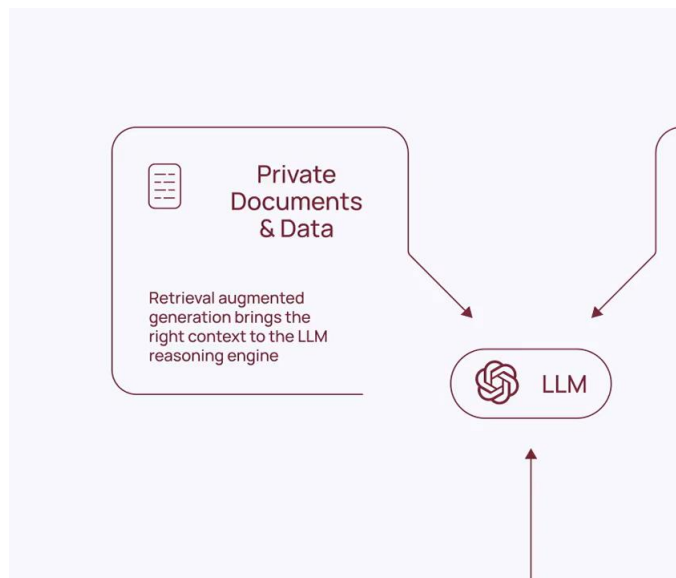


**LangChain** es una **herramienta abierta** que conecta fácilmente **modelos** y bases de **datos**, por ello, facilita la creación de aplicaciones con inteligencia artificial.

## Diseñado para combinarse fácilmente



## Enriquecimiento de datos en tiempo real



## Abierto y neutral

```
>>> from langchain.llms import
>>> from langchain.chains import RetrievalQA
>>> from langchain.vectorstores import

model =
data = .from_documents(...)
chain = RetrievalQA.from_llm(model,
retriever+data.as_retriever())chain.run("...")
```



# 02

Introducción a  
LangChain



# Modelos



Los Modelos de Lenguaje (**LLMs**) son modelos basados en machine learning que pueden **ejecutar tareas** como generación de texto, traducción, resumen o respuesta a preguntas **sin requerir un entrenamiento específico** para cada caso.



## Modelos Chat

Es posible interactuar con los modelos a través de una lista de mensajes los cuales son consumidos por el modelo el cual retorna un mensaje como salida.



## Herramientas

Los modelos ofrecen una forma para interactuar con herramientas (**tool calling**) esto permite crear aplicaciones que puedan comunicarse con servicios externos, APIs y bases de datos.



## Salida estructurada y multimodalidad

Los modelos modernos pueden interactuar con entradas **no textuales** como audios, imágenes o videos. Además, es posible obtener salidas basadas en un esquema, como, por ejemplo, en formato **json**.



# Modelos



Los Modelos de Lenguaje (**LLMs**) son modelos basados en machine learning que pueden **ejecutar tareas** como generación de texto, traducción, resumen o respuesta a preguntas **sin requerir un entrenamiento específico** para cada caso.

Algunos de los parámetros que los modelos actuales implementan:

| Parámetro    | Descripción resumida                                                                                               |
|--------------|--------------------------------------------------------------------------------------------------------------------|
| model        | Nombre o identificador del modelo de IA a usar (por ejemplo, "gpt-3.5-turbo" o "gpt-4").                           |
| temperature  | Controla la aleatoriedad de las respuestas: valores altos generan respuestas más creativas, y bajos, más precisas. |
| timeout      | Tiempo máximo (en segundos) para esperar la respuesta antes de cancelar la solicitud.                              |
| max_tokens   | Límite de tokens en la respuesta; controla la longitud del texto generado.                                         |
| stop         | Secuencias que indican cuándo el modelo debe detener la generación de texto.                                       |
| max_retries  | Número máximo de reintentos en caso de fallos (por ejemplo, por límite de tasa o tiempo de espera).                |
| api_key      | Clave necesaria para autenticarte con el proveedor del modelo.                                                     |
| base_url     | URL del punto de acceso (endpoint) de la API del modelo.                                                           |
| rate_limiter | Limitador de velocidad opcional para evitar exceder los límites de solicitudes.                                    |

# Modelos



Los **proveedores de modelos** (LLM) ponen a disposición del usuario las especificaciones de sus modelos y de las **APIs** que implementan para interactuar con ellos.

## Gemini con LangChain

```
from langchain_google_genai import ChatGoogleGenerativeAI

llm = ChatGoogleGenerativeAI(
    model="gemini-2.5-flash",
    temperature=0,
    max_tokens=None,
    timeout=None,
    max_retries=2,
    # other params...
)
```

## Gemini 2.5 Flash

| Gemini 2.5 Flash                                                                                               |                                                                                                                                                            | Gemini 2.5 Flash Preview | Gemini 2.5 Flash Image | Gemini 2.5 Flash Live        | Gemini 2.5 Flash TTS |
|----------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------|------------------------|------------------------------|----------------------|
| Property                                                                                                       | Description                                                                                                                                                |                          |                        |                              |                      |
|  Model code                 | gemini-2.5-flash                                                                                                                                           |                          |                        |                              |                      |
|  Supported data types       | Inputs<br>Text, images, video, audio                                                                                                                       |                          |                        | Output<br>Text               |                      |
|  Token limits <sup>14</sup> | Input token limit<br>1,048,576                                                                                                                             |                          |                        | Output token limit<br>65,536 |                      |
|  Capabilities               | Audio generation                                                                                                                                           |                          |                        | Batch API                    |                      |
|                                                                                                                | Not supported                                                                                                                                              |                          |                        | Supported                    |                      |
|                                                                                                                | Caching                                                                                                                                                    |                          |                        | Code execution               |                      |
|                                                                                                                | Supported                                                                                                                                                  |                          |                        | Supported                    |                      |
|                                                                                                                | Function calling                                                                                                                                           |                          |                        | Image generation             |                      |
|                                                                                                                | Supported                                                                                                                                                  |                          |                        | Not supported                |                      |
|                                                                                                                | Live API                                                                                                                                                   |                          |                        | Search grounding             |                      |
|                                                                                                                | Not supported                                                                                                                                              |                          |                        | Supported                    |                      |
|                                                                                                                | Structured outputs                                                                                                                                         |                          |                        | Thinking                     |                      |
|                                                                                                                | Supported                                                                                                                                                  |                          |                        | Supported                    |                      |
|                                                                                                                | URL context                                                                                                                                                |                          |                        |                              |                      |
|                                                                                                                | Supported                                                                                                                                                  |                          |                        |                              |                      |
|  Versions                 | Read the <a href="#">model version patterns</a> for more details. <ul style="list-style-type: none"><li>Stable: <a href="#">gemini-2.5-flash</a></li></ul> |                          |                        |                              |                      |
|  Latest update            | June 2025                                                                                                                                                  |                          |                        |                              |                      |
|  Knowledge cutoff         | January 2025                                                                                                                                               |                          |                        |                              |                      |

# Mensajes



Los **mensajes** son la base de comunicación en los modelos de chat, representando tanto la **entrada**, la **salida**, así como el **contexto** o metadatos de la conversación.

**Roles:** Los roles distinguen los tipos de mensajes en una conversación y ayudan al modelo a comprender cómo responder.

| Rol              | Descripción                                                                                                                                                                                                  |
|------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>System</b>    | Se usa para indicar al modelo de chat cómo debe comportarse y proporcionar contexto adicional. No todos los proveedores de modelos de chat lo admiten.                                                       |
| <b>User</b>      | Representa la entrada de un usuario que interactúa con el modelo, generalmente en forma de texto u otra entrada interactiva.                                                                                 |
| <b>Assistant</b> | Representa la respuesta del modelo, que puede incluir texto o una solicitud para invocar herramientas.                                                                                                       |
| <b>Tool</b>      | Se usa para enviar al modelo los resultados de la ejecución de una herramienta después de obtener datos externos o realizar un procesamiento. Se utiliza con modelos que admiten invocación de herramientas. |

# Tipos de Mensajes



Los **mensajes** son la base de comunicación en los modelos de chat, representando tanto la **entrada**, la **salida**, así como el **contexto** o metadatos de la conversación. A continuación, se listan los tres principales:



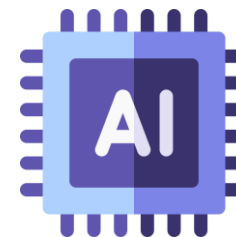
## HumanMessage

Corresponde al rol de usuario. Este tipo de mensaje representa la entrada proveniente de un usuario que interactúa con el modelo.



## SystemMessage

Se utiliza para preparar el comportamiento del modelo e instruirlo para que adopte una persona específica o establecer el tono de la conversación.



## AIMessage

Se utiliza para representar un mensaje con el rol de asistente. Este es el mensaje de respuesta del modelo, que puede incluir texto, una solicitud para invocar herramientas u otro tipo de medios.

# Plantillas prompt



Las **plantillas de prompts** ayudan a convertir la entrada del usuario y los parámetros en instrucciones para el modelo de lenguaje, guiando sus respuestas para que **sean relevantes, coherentes y contextualizadas**.

## String PromptTemplates

Se utilizan para una sola cadena de texto y suelen emplearse en entradas simples.

```
from langchain_core.prompts import PromptTemplate

prompt_template = PromptTemplate.from_template("Tell me a joke about {topic}")

prompt_template.invoke({"topic": "cats"})
```

## ChatPromptTemplates

Se usan para una lista de mensajes. Cada elemento de la lista es una plantilla individual.

```
from langchain_core.prompts import ChatPromptTemplate

prompt_template = ChatPromptTemplate([
    ("system", "You are a helpful assistant"),
    ("user", "Tell me a joke about {topic}")
])

prompt_template.invoke({"topic": "cats"})
```

## MessagesPlaceholder

Se encarga de insertar una lista de mensajes en una posición específica.

```
from langchain_core.prompts import ChatPromptTemplate, MessagesPlaceholder
from langchain_core.messages import HumanMessage, AIMessage

prompt_template = ChatPromptTemplate([
    ("system", "You are a helpful assistant"),
    MessagesPlaceholder("msgs")
])

# Simple example with one message
prompt_template.invoke({"msgs": [HumanMessage(content="hi!")]})

# More complex example with conversation history
messages_to_pass = [
    HumanMessage(content="What's the capital of France?"),
    AIMessage(content="The capital of France is Paris."),
    HumanMessage(content="And what about Germany?")
]

formatted_prompt = prompt_template.invoke({"msgs": messages_to_pass})
print(formatted_prompt)
```

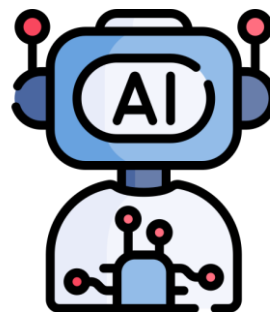
# Cadenas



Las cadenas se refieren a secuencias de **llamadas**, ya sea a un LLM, a una herramienta o a un paso de preprocesamiento de datos. La principal forma de implementación de estas cadenas en LangChain es LangChain Expression Language (**LCEL**)

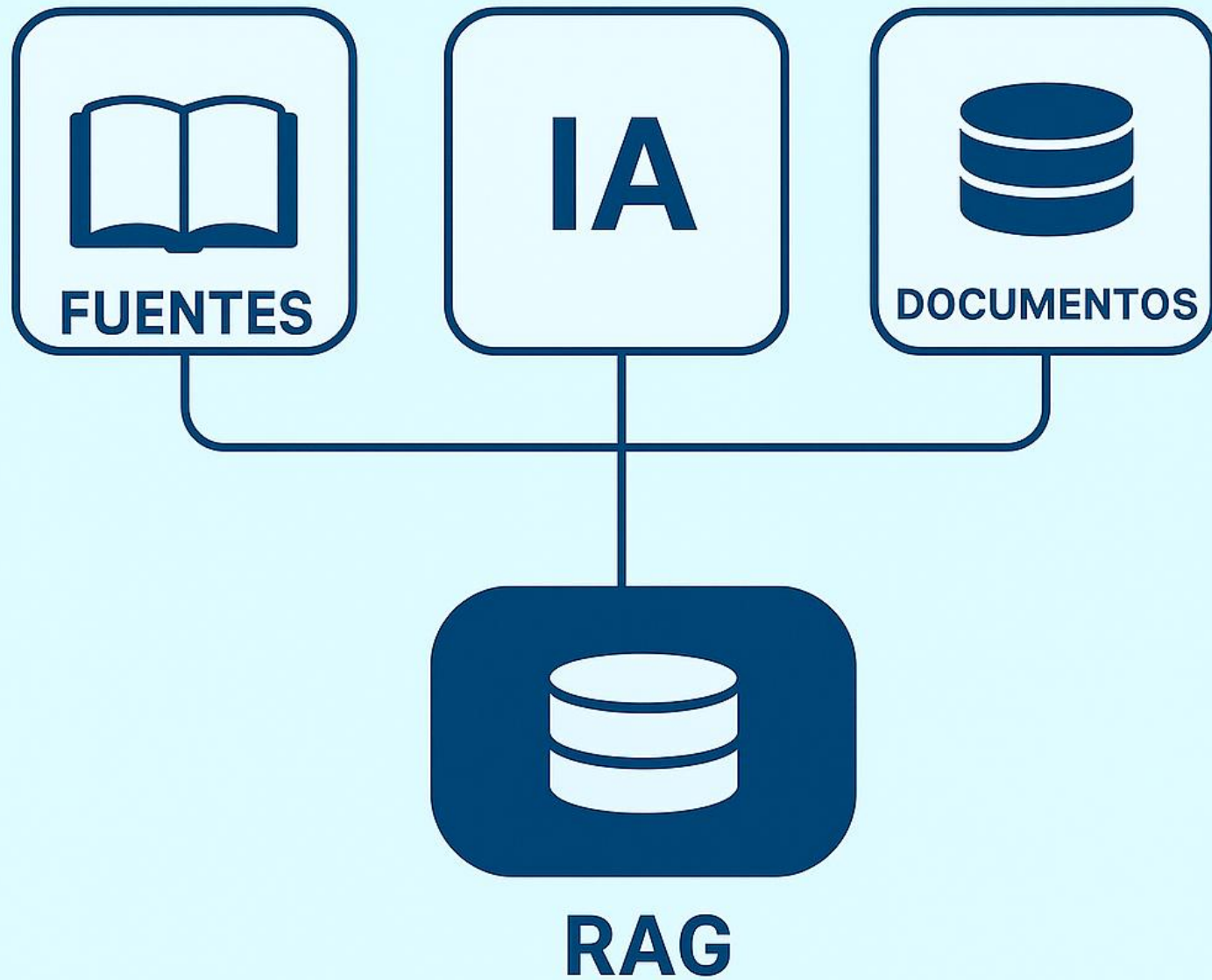


**Plantilla prompt**



**Modelo LLM**

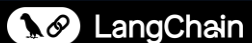




**03**

Herramientas  
RAG: LangChain -  
LlamaIndex

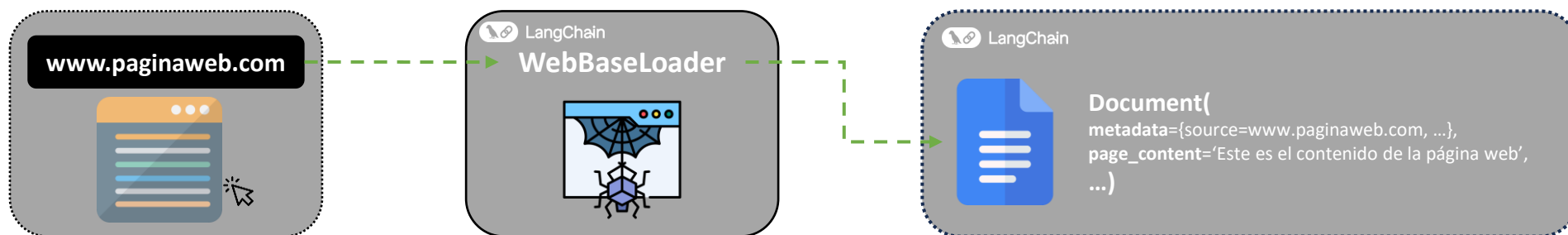




# Cargadores de Documentos



La función de los cargadores de documentos (**document loader**) es transformar información proveniente de distintas fuentes en el **formato estándar** de LangChain: **Document**. Un Document contiene texto y la metadata asociada.





# Cargadores de Documentos



La función de los cargadores de documentos es transformar información proveniente de distintas fuentes en el **formato estándar** de LangChain: **Document**. Un Document contiene texto y la metadata asociada.

Se listan algunos de los principales cargadores de documentos:

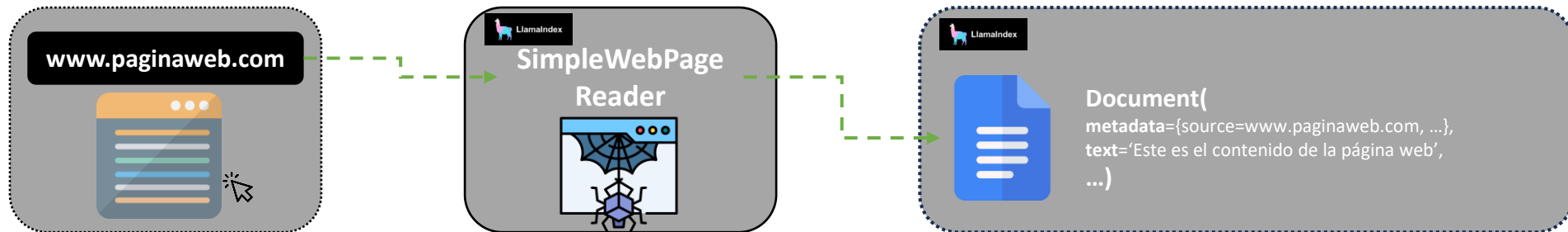
| Cargador documento | Descripción                                                                        |
|--------------------|------------------------------------------------------------------------------------|
| Web                | Utiliza urllib y BeautifulSoup para cargar y analizar páginas web en formato HTML. |
| PyPDF              | Utiliza pypdf para cargar y analizar archivos PDF.                                 |
| CSVLoader          | Carga archivos csv                                                                 |
| DirectoryLoader    | Todos los archivos en un directorio determinado.                                   |



# Lectores de Documentos



La función de los lectores de documentos (**document readers**) es transformar información proveniente de distintas fuentes en el **formato estándar** de LlamaIndex: **Document**. Un Document contiene texto y la metadata asociada.



La **interface** es similar a la  
de **LangChain**



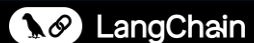
# Lectores de Documentos



La función de los lectores de documentos (**document readers**) es transformar información proveniente de distintas fuentes en el **formato estándar** de LlamaIndex: **Document**. Un Document contiene texto y la metadata asociada.

Se listan algunos de los principales lectores de documentos:

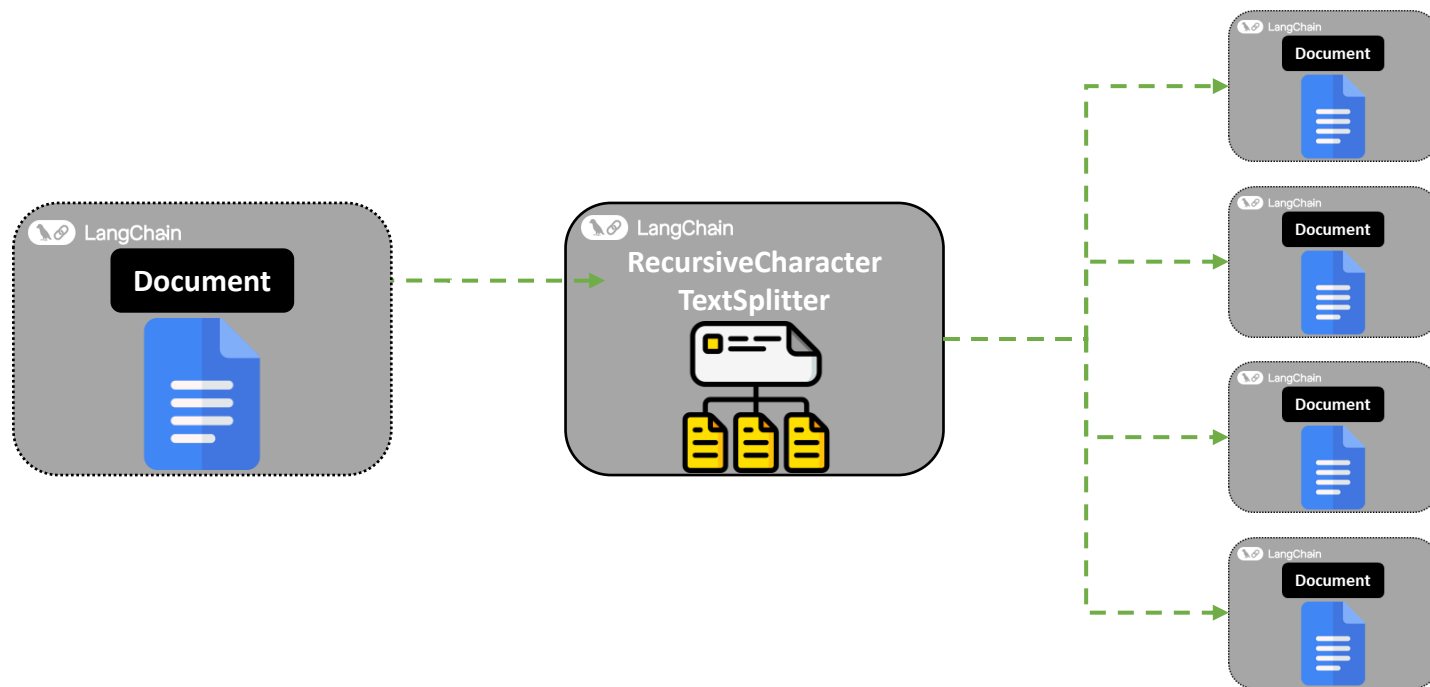
| Cargador documento    | Descripción                                                        |
|-----------------------|--------------------------------------------------------------------|
| SimpleWebPageReader   | Carga y extrae texto de páginas web.                               |
| PDFReader             | Lee y convierte el contenido de archivos PDF en texto.             |
| PandasCSVReader       | Importa archivos CSV y los transforma en documentos usando pandas. |
| SimpleDirectoryReader | Todos los archivos en un directorio determinado.                   |

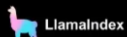


## Divisores de texto



La división de documentos consiste en **fragmentar textos** extensos en partes más pequeñas, lo que facilita un procesamiento más uniforme, **supera las limitaciones** de tamaño de entrada de los modelos y mejora la calidad de las representaciones textuales.

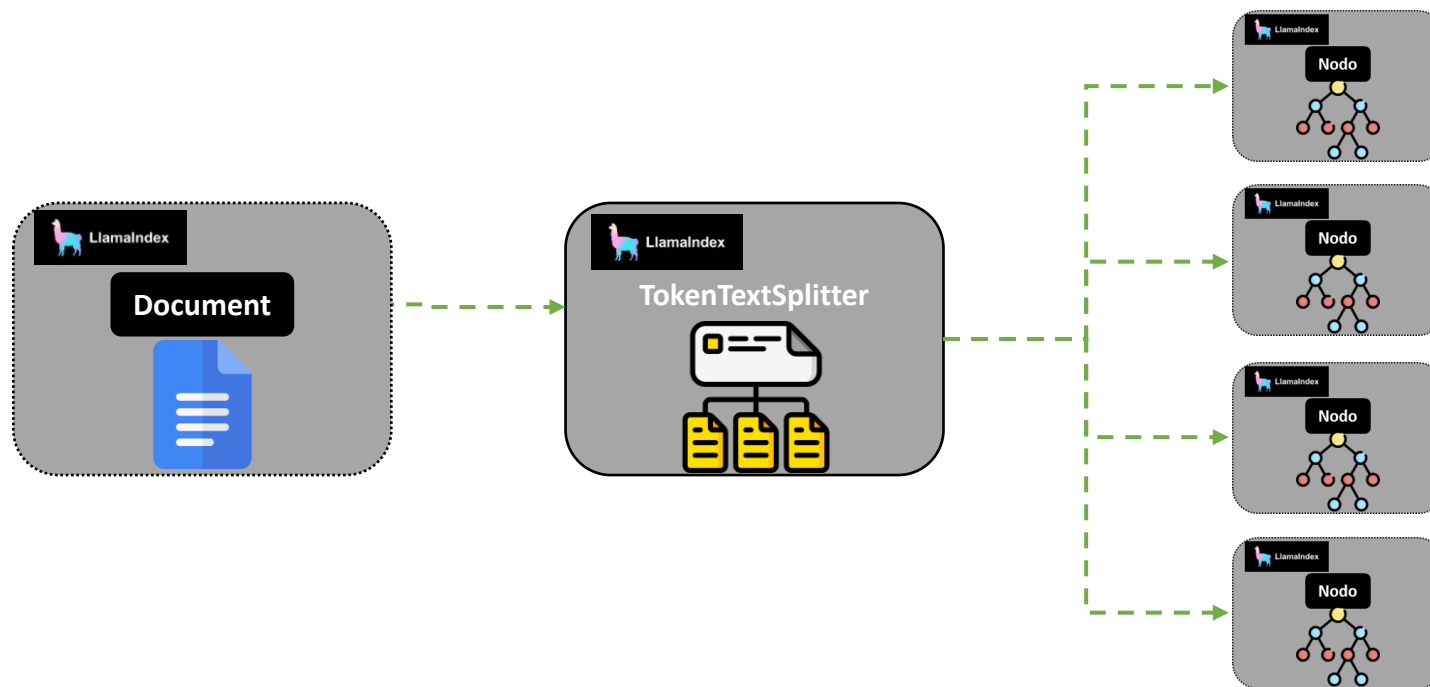




## Divisores de texto



En LlamaIndex, los documentos suelen dividirse en nodos, que son fragmentos más pequeños que facilitan su manejo y preservan su estructura interna. Esta división puede hacerse de forma automática mediante text splitters (para texto simple) o node parsers (para formatos más complejos).



La **interface** es similar a la  
de **LangChain**

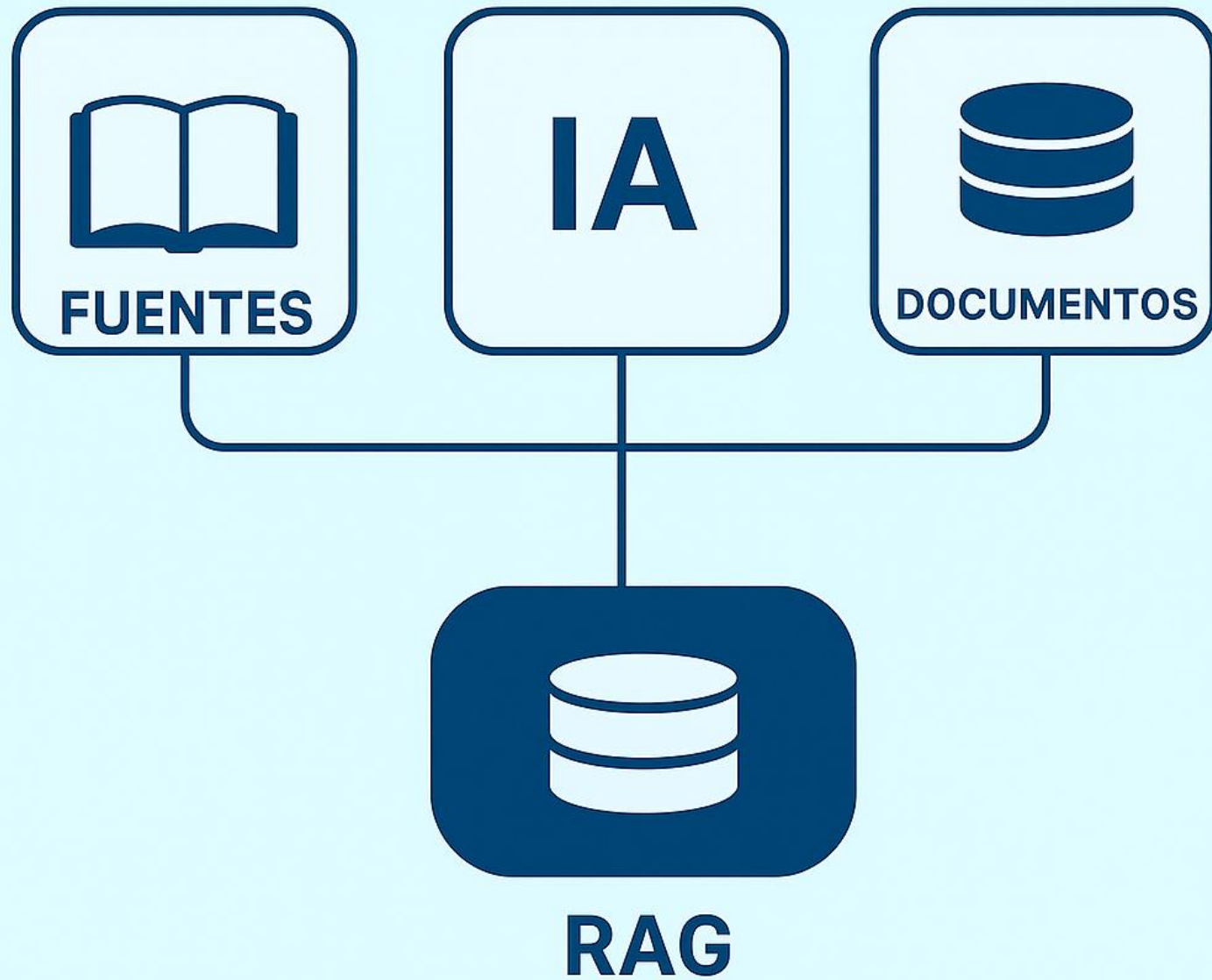
# Divisores de texto



La división de documentos consiste en **fragmentar textos** extensos en partes más pequeñas, lo que facilita un procesamiento más uniforme, **supera las limitaciones** de tamaño de entrada de los modelos y mejora la calidad de las representaciones textuales.

Se listan algunos de los principales enfoques para la división de documentos:

| Enfoque                               | Descripción                                                                                                               |
|---------------------------------------|---------------------------------------------------------------------------------------------------------------------------|
| Basado en longitud                    | Divide el texto según un umbral de tamaño (tokens o caracteres)                                                           |
| Basado en estructura de texto         | Respetar unidades lingüísticas como párrafos y oraciones; divide recursivamente si excede el tamaño                       |
| Basado en la estructura del documento | Usa la estructura inherente del documento (HTML, Markdown, JSON) para dividir por secciones, encabezados, etiquetas, etc. |
| Basado en significado semántico       | Analiza el contenido con embeddings / semántica para hallar puntos de división coherentes                                 |



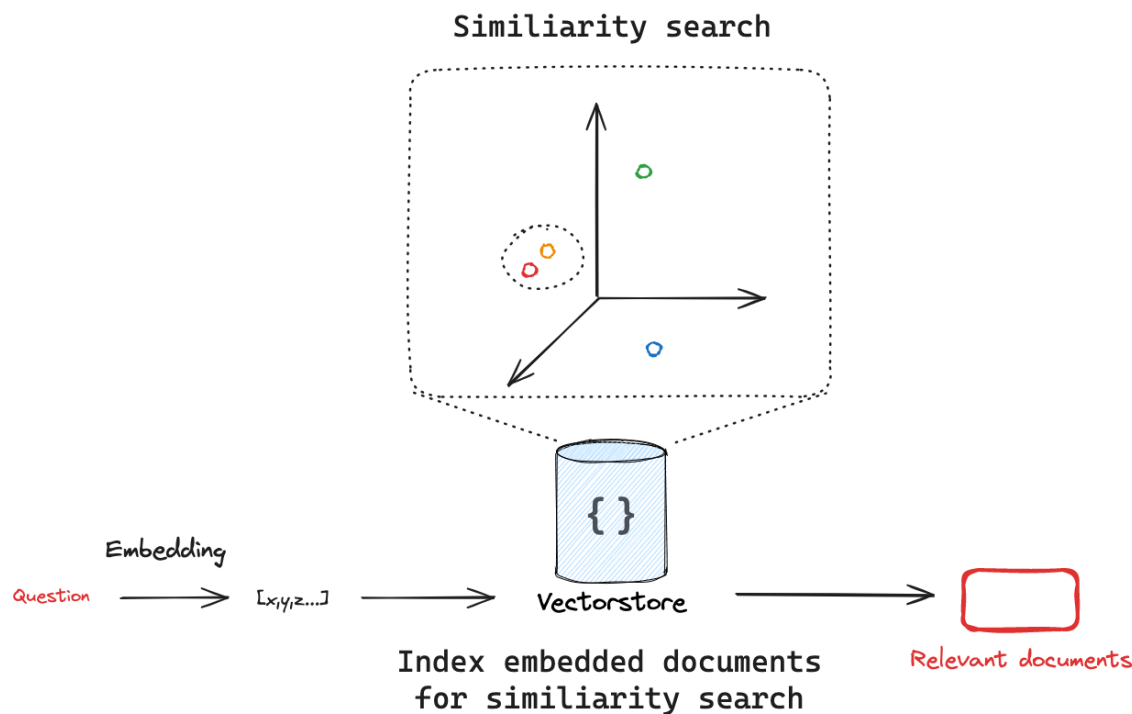
**04**  
Herramientas  
RAG: Vector  
Stores



# Vector Store



Los almacenamientos vectoriales permiten **buscar información** en datos no estructurados mediante representaciones semánticas (**embeddings**), recuperando resultados por similitud en lugar de coincidencias exactas.



# Vector Store



Los almacenamientos vectoriales permiten buscar información en datos no estructurados mediante representaciones semánticas (embeddings), recuperando resultados por similitud en lugar de coincidencias exactas.

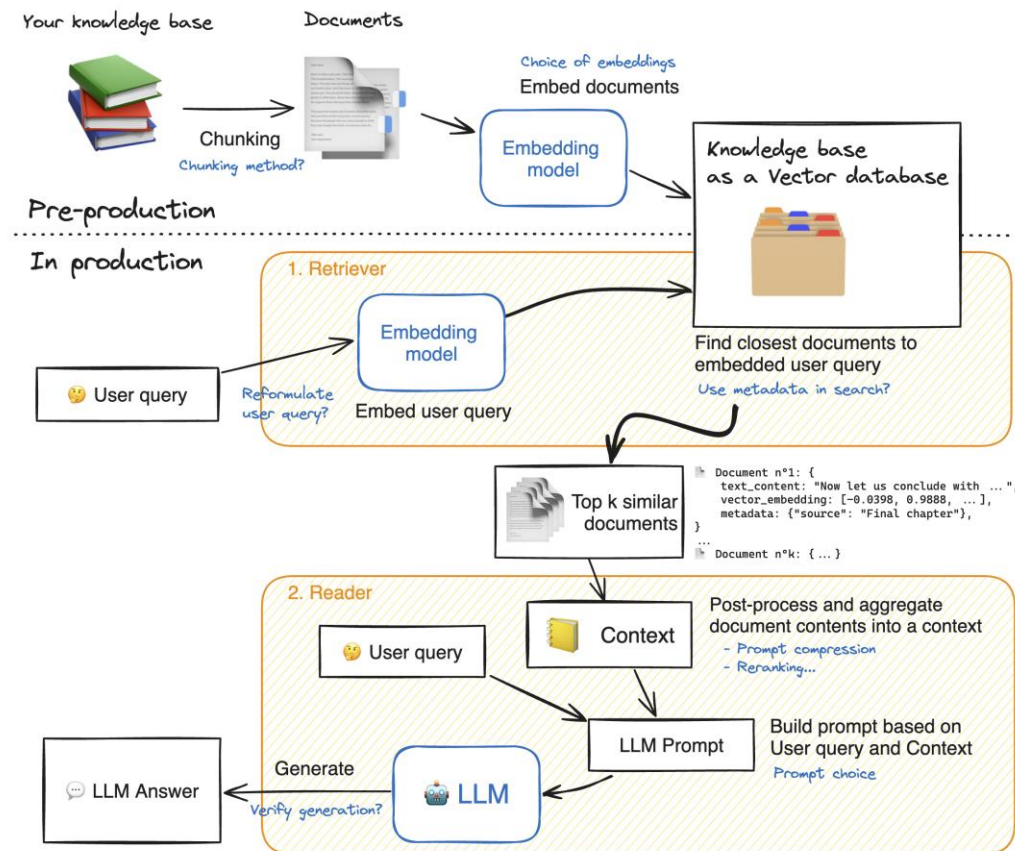
Se listan algunos de los principales vector stores:

| Vector Store | Descripción breve                                                                                            |
|--------------|--------------------------------------------------------------------------------------------------------------|
| Chroma       | Base de datos vectorial ligera y de código abierto, ideal para entornos locales y prototipos.                |
| FAISS        | Biblioteca de Meta optimizada para búsquedas de similitud en memoria, muy rápida y eficiente.                |
| Pinecone     | Servicio en la nube totalmente gestionado, escalable y con alto rendimiento.                                 |
| PGVector     | Extensión de PostgreSQL que permite almacenar y consultar embeddings dentro de una base de datos relacional. |

## Retrieval-Augmented Generation (RAG)



“La **Generación Aumentada con Recuperación (RAG)** es el proceso de optimizar la salida de un modelo de lenguaje de gran tamaño, de manera que haga referencia a una base de conocimiento autorizada externa a sus fuentes de datos de entrenamiento antes de generar una respuesta.” AWS





## Participación

1. Elijan una aplicación o problema (por ejemplo, un chatbot de soporte interno, un resumidor de documentos legales, un asistente para codificación).
2. Identifiquen tres herramientas o componentes clave de LangChain que serían esenciales para su solución.
3. Tendrán 20 minutos para crear la presentación y 5 minutos para exponer.



Facultad de  
Ingeniería  
Económica,  
Estadística y  
Ciencias  
Sociales

Centro de  
Formación  
Continua

**II PROGRAMA DE ESPECIALIZACIÓN EN IA  
GENERATIVA Y MACHINE LEARNING OPS**

# Gracias